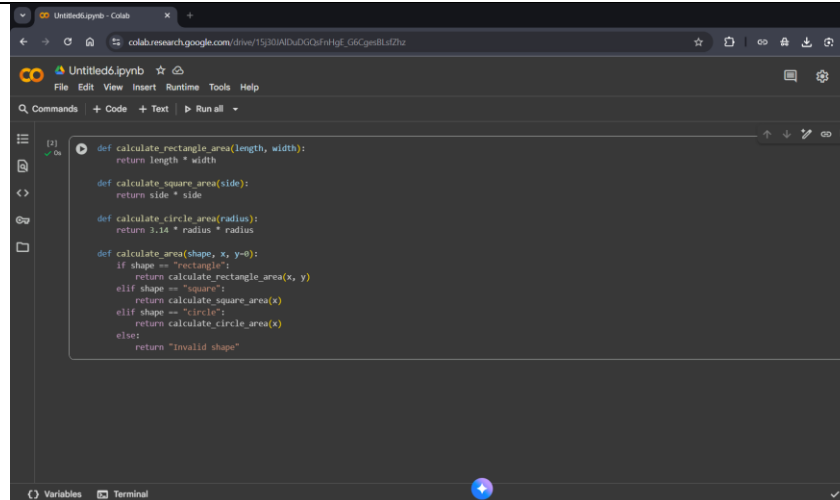


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr. J. Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S. Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch. Rajitha	
		Mr. M Prakash	
		Mr. B. Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week 7 - Wednesday	Time(s)	
Duration	2 Hours	Applicable to Batches	
Assignment Number: 13.3 (Present assignment number) / 24 (Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 13 – Code Refactoring: Improving Legacy Code with AI Suggestions Lab Objectives To introduce the concept of code refactoring and why it matters (readability, maintainability, performance).		Week 5 - Monday

	• To practice using AI tools for identifying and suggesting improvements in legacy code. • To evaluate the before vs. after versions for clarity, performance, and correctness. • To reinforce responsible AI-assisted coding practices (avoiding over-reliance, validating outputs). Learning Outcomes After completing this lab, students will be able to: 1. Use AI to analyze and refactor poorly written Python code. 2. Improve code readability, efficiency, and error handling. 3. Document AI-suggested improvements through comments and explanations. 4. Apply refactoring strategies without changing functionality. 5. Critically reflect on AI's refactoring suggestions. Task Description #1 – Remove Repetition Task: Provide AI with the following redundant code and ask it to refactor Python Code <pre>def calculate_area(shape, x, y=0): if shape == "rectangle": return x * y elif shape == "square": return x * x elif shape == "circle": return 3.14 * x * x</pre> Expected Output • Refactored version with dictionary-based dispatch or separate functions. • Cleaner and modular design.	
--	---	--

A screenshot of a Google Colab notebook interface. The browser address bar shows a URL from colab.research.google.com. The notebook is titled 'Untitled6.ipynb'. The code editor contains the following Python code:

```
[2] def calculate_rectangle_area(length, width):  
    return length * width  
  
def calculate_square_area(side):  
    return side * side  
  
def calculate_circle_area(radius):  
    return 3.14 * radius * radius  
  
def calculate_area(shape, x, y=0):  
    if shape == "rectangle":  
        return calculate_rectangle_area(x, y)  
    elif shape == "square":  
        return calculate_square_area(x)  
    elif shape == "circle":  
        return calculate_circle_area(x)  
    else:  
        return "Invalid shape"
```

The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with icons for commands, code, text, and running, and a bottom panel with tabs for Variables and Terminal.

Task Description #2 – Error Handling in Legacy Code

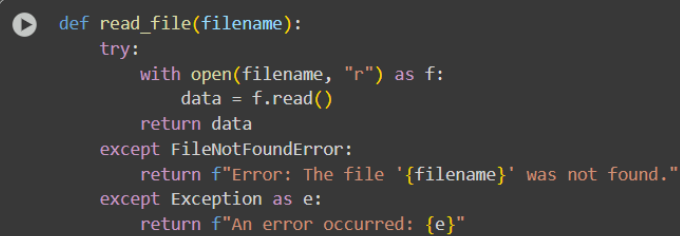
Task: Legacy function without proper error handling

Python Code

```
def read_file(filename):  
    f = open(filename, "r")  
    data = f.read()  
    f.close()  
    return data
```

Expected Output:

AI refactors with with open() and try-except:

A screenshot of a code editor showing the refactored Python code. The code is as follows:

```
def read_file(filename):  
    try:  
        with open(filename, "r") as f:  
            data = f.read()  
        return data  
    except FileNotFoundError:  
        return f"Error: The file '{filename}' was not found."  
    except Exception as e:  
        return f"An error occurred: {e}"
```

Task Description #3 – Complex Refactoring

Task: Provide this legacy class to AI for readability and modularity improvements:

Python Code

class Student:

```
def __init__(self, n, a, m1, m2, m3):
    self.n = n
    self.a = a
    self.m1 = m1
    self.m2 = m2
    self.m3 = m3
def details(self):
    print("Name:", self.n, "Age:", self.a)
def total(self):
    return self.m1+self.m2+self.m3
```

Expected Output:

- AI improves naming (name, age, marks).
- Adds docstrings.
- Improves print readability.
- Possibly uses `sum(self.marks)` if marks stored in a list.

```
class Student:
    """
    Represents a student with name, age, and marks in three subjects.
    """
    def __init__(self, name, age, mark1, mark2, mark3):
        """
        Initializes a Student object.

        Args:
            name (str): The name of the student.
            age (int): The age of the student.
            mark1 (int): The mark in the first subject.
            mark2 (int): The mark in the second subject.
            mark3 (int): The mark in the third subject.
        """
        self.name = name
        self.age = age
        self.marks = [mark1, mark2, mark3]

    def details(self):
        """
        Prints the name and age of the student.
        """
        print(f"Name: {self.name}, Age: {self.age}")

    def total(self):
        """
        Calculates and returns the total marks of the student.
        """
        return sum(self.marks)
```

Task Description #4 – Inefficient Loop Refactoring

Task: Refactor this inefficient loop with AI help

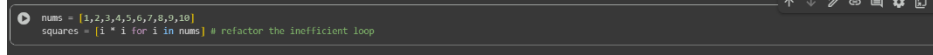
Python Code

```
nums = [1,2,3,4,5,6,7,8,9,10]
```

```
squares = []
```

```
for i in nums:
```

```
    squares.append(i * i)
```



```
nums = [1,2,3,4,5,6,7,8,9,10]  
squares = [i * i for i in nums] # refactor the inefficient loop
```

Expected Output: AI suggested a **list comprehension**